Title

Convolutional Neural Networks: Feature Extraction using Filters, Stride, and Pooling.

1. Introduction

Deep learning is a rapidly growing area within machine learning that focuses on training neural networks with multiple layers to learn complex patterns from data. In traditional machine learning approaches, a large amount of effort is often spent on manually designing features that help the model understand the data. While this can work well for simpler problems, it becomes time-consuming and less effective when dealing with complex data such as images. This is because images contain a large amount of information, including spatial relationships and hierarchical patterns, which are difficult to capture using manually engineered features.

Unlike tabular data, where each feature is usually independent, image data is highly structured. The value of a pixel is closely related to the values of its neighboring pixels. For instance, groups of nearby pixels combine to form edges, textures, and shapes, which are essential for understanding the content of an image. Traditional machine learning models often fail to capture these spatial relationships effectively, leading to lower performance in image-based tasks. This limitation highlights the need for more advanced models that can process images in a way that preserves their structure.

Convolutional Neural Networks (CNNs) are specifically designed to address this challenge. They are capable of analysing images by automatically learning important features directly from raw pixel values, without requiring manual feature extraction. Through multiple layers, CNNs gradually build an understanding of the image, starting with simple features like edges and progressing to more complex patterns such as shapes and objects. As highlighted in the lecture material, CNNs are particularly powerful because they learn hierarchical representations, where lower layers focus on basic features and deeper layers capture more abstract concepts .

This tutorial aims to provide a clear understanding of how CNNs perform feature extraction. It focuses on key concepts such as convolution, filters, stride, and pooling, explaining both how they work and why they are important. By the end of this tutorial, readers will have a solid understanding of CNNs and their effectiveness in solving image-related problems.

2. What is a Convolutional Neural Network?

A Convolutional Neural Network (CNN) is a specialised type of deep neural network that is designed to process structured data, particularly images. Unlike traditional fully connected neural networks, where every neuron is connected to every other neuron, CNNs take advantage of the spatial structure present in images. This allows them to process visual data more efficiently and effectively.

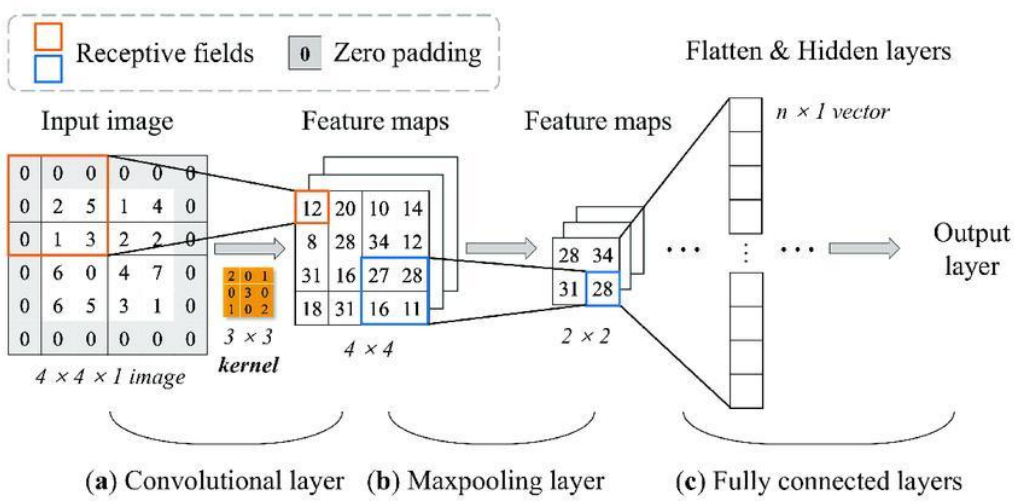
A typical CNN architecture is made up of several key components that work together to analyse and interpret images:

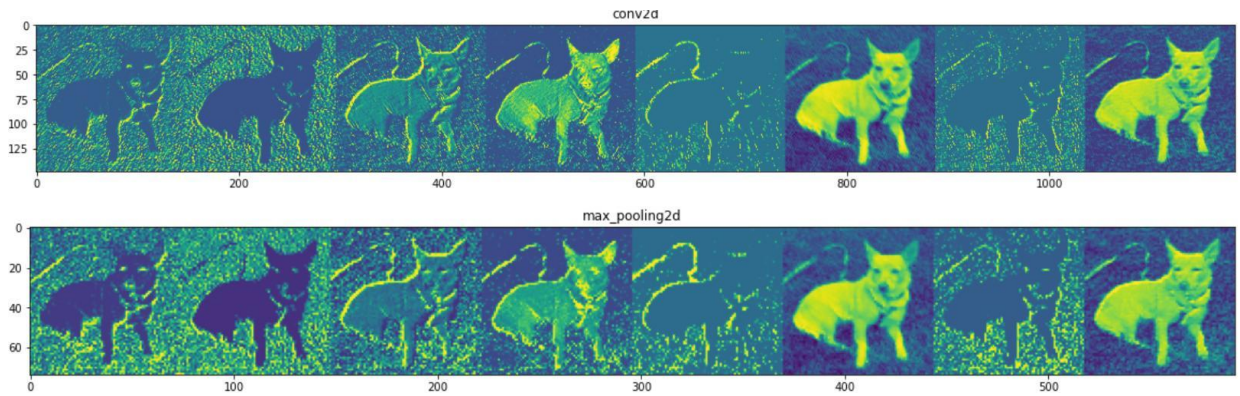
- Convolutional Layers: These layers are responsible for extracting features from the input image by applying filters.
- Activation Functions (e.g., ReLU): These introduce non-linearity, allowing the network to learn complex patterns.
- Pooling Layers: These reduce the size of the feature maps, making the model more efficient.
- Fully Connected Layers: These use the extracted features to perform classification or prediction.

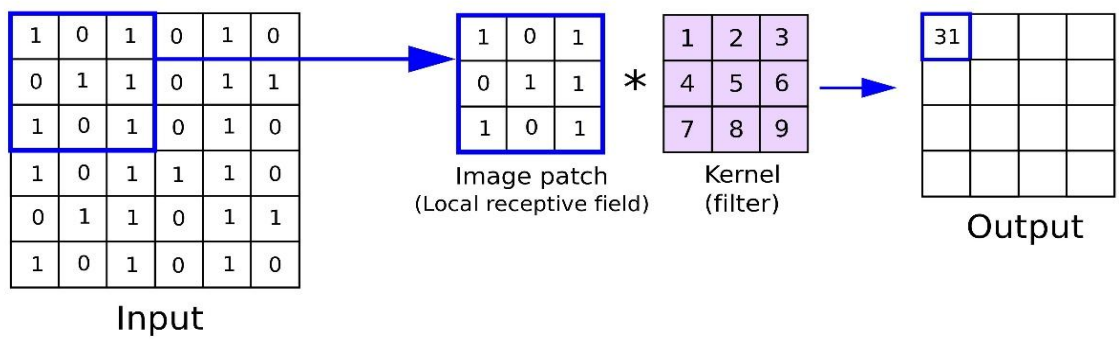
The most important idea behind CNNs is automatic feature learning. Instead of relying on humans to decide which features are important, the network learns this information directly from the data during training. In the early stages of the network, simple features such as edges and lines are detected. As the data passes through deeper layers, these simple features are combined to form more complex representations, such as shapes, textures, and eventually entire objects.

This layered learning approach is known as hierarchical feature learning and is one of the key reasons CNNs are so powerful. By learning features at different levels of abstraction, CNNs are able to capture both low-level and high-level information in images. This makes them significantly more effective than traditional machine learning models, particularly in tasks such as image classification, object detection, and facial recognition.

3. Convolution Operation







The convolution operation is the fundamental building block of a CNN. It involves applying a small matrix called a filter (or kernel) to an input image in order to extract features.

The filter slides across the image both horizontally and vertically. At each position, the filter performs element-wise multiplication with the corresponding region of the image and then sums the results. This produces a single value in the output feature map.

Mathematically, the operation can be expressed as:

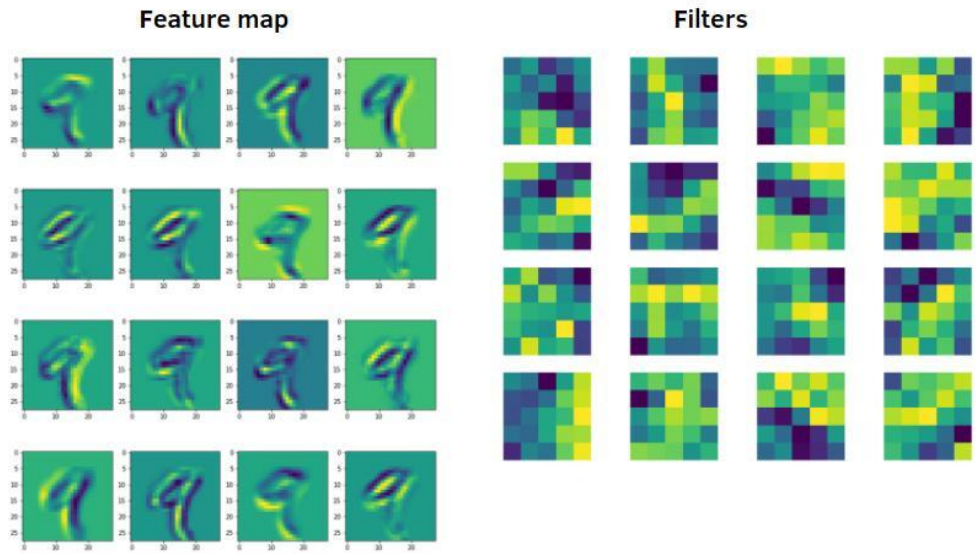
$$Output = \sum(Image \times Filter)$$

From the lecture slides, convolution works by sliding a kernel across the image and computing the dot product between the filter and the image region.

This process is repeated across the entire image, resulting in a new matrix known as a feature map. Each value in this feature map represents how strongly a particular feature is detected in that region.

An important aspect of convolution is that the same filter is applied across the entire image. This is known as parameter sharing, and it significantly reduces the number of parameters in the model, making CNNs more efficient than fully connected networks.

4. Filters and Feature Extraction



Filters (or kernels) are responsible for detecting features in an image. Each filter is designed to respond to a specific pattern, such as edges, lines, or textures.

For example:

- Edge detection filters highlight boundaries between objects
- Sharpening filters enhances details
- Identity filters preserve the original image

The output of applying a filter is called a feature map, which shows where that feature is present in the image.

As stated in the lecture slides, “which features are extracted depends on the filter used” . During training, CNNs automatically learn the optimal filters needed for a given task.

In deep CNNs:

- Early layers detect simple features (edges, lines)
- Middle layers detect patterns (corners, textures)
- Deeper layers detect complex features (objects, faces)

This hierarchical feature extraction allows CNNs to understand images in a way similar to human visual perception.

5. Stride and Output Size

Stride refers to the number of pixels the filter moves across the image at each step during the convolution operation.

- Stride = 1: Filter moves one pixel at a time → detailed output
- Stride = 2 or more: Filter skips pixels → smaller output

The output size of a convolution can be calculated using the formula:

$$Output = \frac{N - F}{Stride} + 1$$

Where:

- N= input size
- F= filter size

For example:

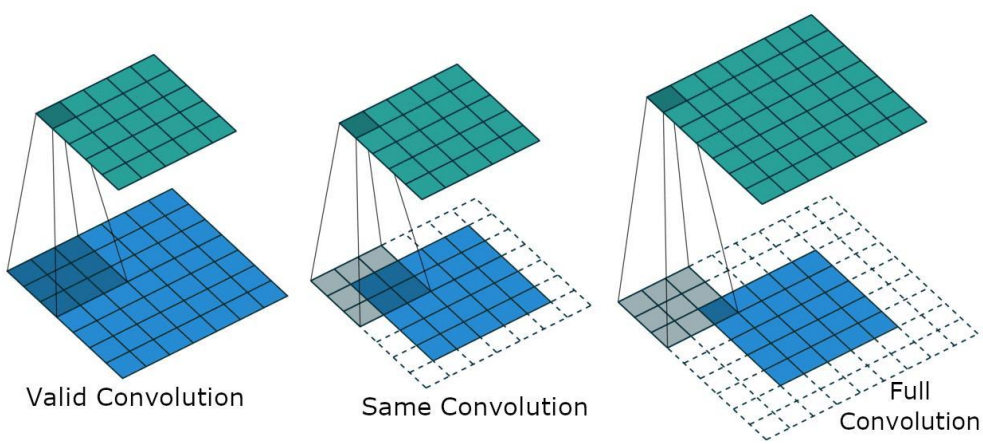
- Input = 7×7
- Filter = 3×3
- Stride = 1

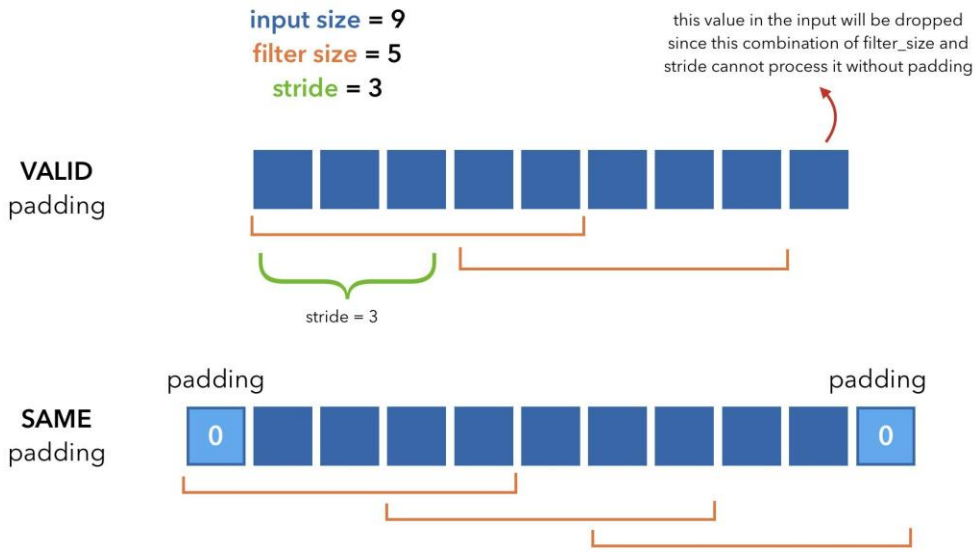
$$Output = \frac{7 - 3}{1} + 1 = 5$$

So, the output feature map is 5×5.

Increasing the stride reduces the spatial dimensions of the output, which can help reduce computational cost. However, larger strides may also lead to loss of information.

6. Padding





Padding is the process of adding extra pixels (usually zeros) around the border of the input image before applying convolution.

Without padding, repeated convolution operations reduce the size of the image. This can lead to loss of important information, especially at the edges. Padding helps to address this issue.

The output size with padding is given by:

$$Output = \frac{N - F + 2P}{Stride} + 1$$

Where:

- P= padding size

For example:

- Input = 7×7

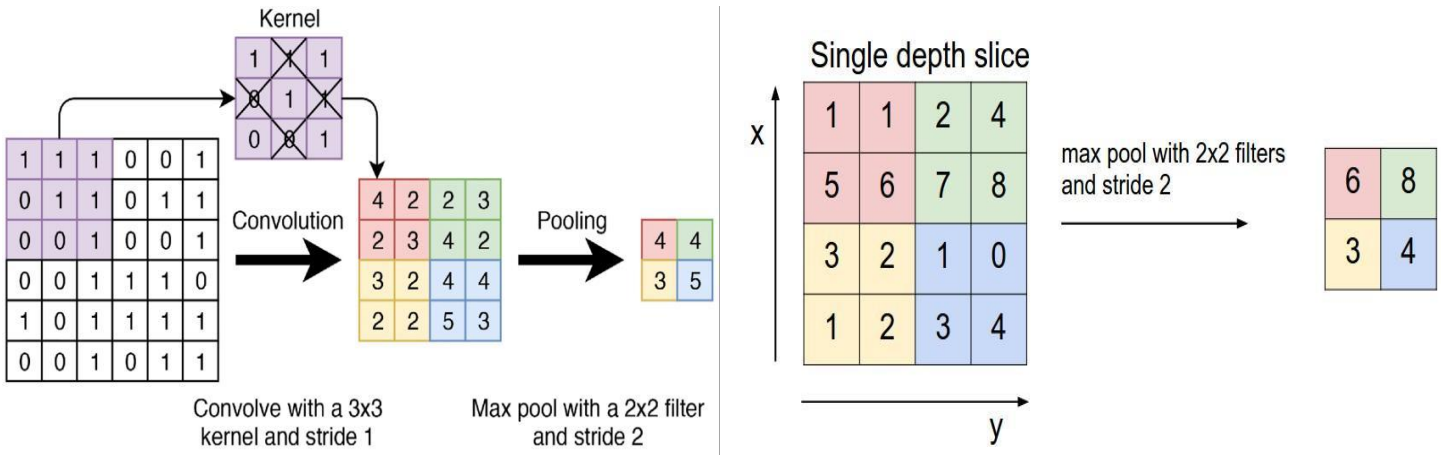
- Filter = 3×3
- Padding = 1
- Stride = 1

$$Output = \frac{7 - 3 + 2(1)}{1} + 1 = 7$$

Thus, padding allows us to preserve the original image size.

According to the lecture slides, padding helps preserve spatial dimensions and prevents loss of edge information.

7. Pooling Layer



Pooling layers are used to reduce the spatial dimensions of feature maps while retaining important information.

The most common types of pooling are:

Max Pooling

Selects the maximum value in each region and captures the most important feature.

Average Pooling

Computes the average value and preserves general information.

Pooling provides several benefits:

- Reduces computational complexity.

- Helps prevent overfitting
- Makes the model invariant to small translations

As noted in the lecture slides, pooling reduces spatial dimensions while retaining the most important information.

8. CNN Architecture

A typical CNN processes data through a sequence of layers:

Input Image → Convolution → ReLU → Pooling → Fully Connected → Output

- **Convolution:** Extracts features
- **ReLU:** Introduces non-linearity
- **Pooling:** Reduces dimensions
- **Fully Connected:** Performs classification

By stacking multiple convolution and pooling layers, CNNs learn increasingly complex features.

Practical Example (MNIST Dataset)

To demonstrate CNNs in practice, we use the MNIST dataset, which contains 28×28 grayscale images of handwritten digits (0–9).

The process involves:

1. Loading the dataset
2. Normalizing pixel values
3. Building a CNN model
4. Training the model
5. Evaluating performance

The CNN learns to identify digits by extracting features such as edges and shapes. For example, it may learn to detect curves for digits like “6” or straight lines for digits like “1”.

9. Code Overview

The implementation uses Keras, a high-level deep learning API.

The model includes:

- Convolutional layers for feature extraction

- Pooling layers for down sampling
- Dense layers for classification

The full implementation is provided in the accompanying Jupyter Notebook.

10.Conclusion

Convolutional Neural Networks are a powerful tool for image processing tasks. Their ability to automatically learn hierarchical features makes them significantly more effective than traditional machine learning methods for visual data.

This tutorial explored how CNNs extract features using convolution, filters, stride, and pooling. Understanding these components is essential for designing and implementing deep learning models.

CNNs are widely used in real-world applications such as medical imaging, autonomous vehicles, and facial recognition. As deep learning continues to evolve, CNNs remain a fundamental component of modern artificial intelligence systems.

11.References

- Lecture Slides: Machine Learning and Neural Networks
- Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*
- TensorFlow Documentation
- Towards Data Science